

## Maquina de bolas

Madina ha inventado una máquina lanzapelotas para entretener a los participantes de la IOI. La estructura interna de la máquina es la siguiente:

- La máquina consta de  $N$  **nodos**, indexados desde 0 hasta  $N - 1$ . El nodo  $N - 1$  se denomina **raíz**.
- Para cada nodo  $u$  con  $0 \leq u < N - 1$ , el **padre** del nodo  $u$  es un nodo  $P[u]$  con  $P[u] > u$ , y el nodo  $u$  es un **hijo** de  $P[u]$ . La raíz no tiene padre. Un nodo sin hijos se denomina **hoja**.

**No** conoces  $N$  ni el padre  $P[u]$  de ningún nodo  $u$ . En cambio, Madina te dice  $M$ , el número de hojas, y que los índices de las hojas son  $0, 1, \dots, M - 1$ . Tu tarea es determinar la estructura interna de la máquina al operarla.

Cada nodo de la máquina puede contener como máximo una **bola**, y cada bola tiene un **valor** que es un entero no negativo. Decimos que un nodo tiene valor  $x$  si contiene una bola con valor  $x$ . Un nodo está **ocupado** si contiene una bola; de lo contrario, está **libre**. Inicialmente, todos los nodos están libres.

Puedes realizar dos tipos de operaciones:

- **insertar**( $U, X$ ): escribe el valor  $X$  en una nueva bola e intenta insertarla en la hoja  $U$ .
  - Si el nodo hoja  $U$  está ocupado, la operación devuelve `false` sin insertar la bola.
  - Si el nodo hoja  $U$  está libre, la bola se coloca en el nodo  $U$ . Luego, la bola se desplaza repetidamente hacia el nodo padre de su nodo actual, siempre y cuando ese nodo padre esté libre. El desplazamiento se detiene cuando la bola llega a la raíz o a un nodo cuyo padre esté ocupado. La operación devuelve `true`.
- **collect**(): si la raíz está libre (lo que significa que la máquina está vacía), `collect` devuelve un array vacío. De lo contrario, la función recursiva `traverse` descrito a continuación recopila los valores de todas las bolas que se encuentran actualmente en la máquina en un array  $S$ . Inicialmente, el array  $S$  está vacío y se llama a `traverse(N - 1)`.

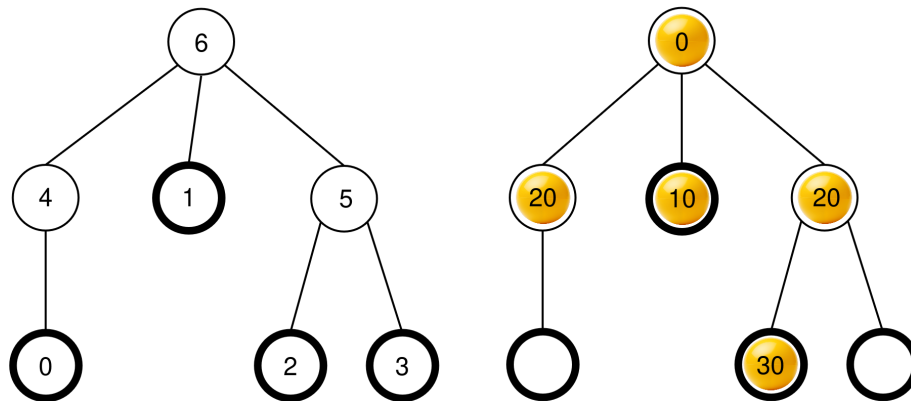
```

traverse(u):
    añade el valor de la bola del nodo u al final de S
    sea c[u] la lista de los hijos ocupados de u
    ordena c[u] en orden no-decreciente de acuerdo a los valores
        de las bolas de esos nodos (si varios nodos tienen el mismo
            valor, ellos pueden aparecer en cualquier orden)
    para cada v en c[u]:
        traverse(v)

```

Después de que esta función termina, **se remueven todas las bolas** de la máquina y `collect` retorna  $S$ .

Por ejemplo, considere una máquina con  $N = 7$  nodos y arreglo de padres  $P = [4, 6, 5, 5, 6, 6]$ . La figura de la izquierda muestra la máquina vacía con los índices de los nodos, mientras que la figura de la derecha muestra una configuración posible después de alguna secuencia de operaciones `insert` (esta secuencia de operaciones se da en la Sección Ejemplo).



Cuando se llama `collect()`, la raíz (nodo 6) está ocupada, entonces  $S$  se inicializa como  $S = []$  y se llama `traverse(6)`.

**traverse(6):** el nodo 6 tiene valor 0; añadiéndolo a  $S$  se obtiene  $S = [0]$ . Los hijos del nodo 6 son 4, 1, y 5, todos los cuales están ocupados. Sus valores son 20, 10, y 20, respectivamente. Ordenando en orden no-decreciente se produce  $c[6] = [1, 5, 4]$  (Note que  $c[6] = [1, 4, 5]$  también sería válido, ya que los nodos 4 y 5 tienen el mismo valor). Posteriormente, la función llama `traverse` en cada nodo de  $c[6]$  en ese orden.

- **traverse(1):** el nodo 1 tiene valor 10; añadiéndolo a  $S$  se obtiene  $S = [0, 10]$ . El nodo 1 no tiene hijos ocupados, entonces esta llamada termina.
- **traverse(5):** el nodo 5 tiene valor 20; añadiéndolo a  $S$  se obtiene  $S = [0, 10, 20]$ . El nodo 5 tiene un hijo ocupado, el nodo 2, entonces  $c[5] = [2]$  y la función llama a `traverse(2)`.
  - **traverse(2):** el nodo 2 tiene valor 30; añadiéndolo a  $S$  se obtiene  $S = [0, 10, 20, 30]$ . El nodo 2 no tiene hijos ocupados, entonces esta llamada termina.

- La ejecución se devuelve a `traverse(5)`, la cual ha procesado todos los hijos en `c[5]`, entonces este llamado termina.
- **`traverse(4)`**: el nodo 4 tiene valor 20; añadiéndolo a  $S$  se obtiene  $S = [0, 10, 20, 30, 20]$ . El nodo 4 no tiene hijos ocupados, entonces este llamado termina.

Todas las llamadas se han terminado y no hay más nodos que procesar.

Finalmente, todas las bolas son removidas de la máquina y `collect` retorna el arreglo  $S = [0, 10, 20, 30, 20]$ .

Su tarea es determinar el número de nodos  $N$  y reportar un arreglo de padres  $R = [R[0], R[1], \dots, R[N-2]]$  que describa la estructura de la máquina, pero posiblemente con una numeración diferente de los nodos que no son hojas y no son la raíz. Formalmente, un arreglo reportado  $R$  se considera correcto si es posible asignar una etiqueta diferente  $L[u]$  a cada nodo  $u$  con  $0 \leq L[u] < N$  tal que:

- $L[u] = u$  para todo  $0 \leq u < M$  y  $u = N-1$ , y
- $L[P[u]] = R[L[u]]$  para todo  $0 \leq u < N-1$ .

**No** se requiere que  $R[u] > u$ . La máquina **debe estar vacía** cuando usted reporte su solución.

Sea  $K$  el número de operaciones `collect` realizadas y  $B$  el valor máximo escrito en cualquier bola a lo largo de todos los llamados `insert`. Sea  $C = K + B$ .  $C$  **no debe exceder** 1000. Su puntaje en algunas subtarefas depende del valor de  $C$ .

## Detalles de implementación

Usted debe implementar la siguiente función.

```
std::vector<int> find_structure(int M)
```

- $M$ : El número de hojas en la máquina.
- La función será llamado exactamente una vez por cada caso de prueba.
- La función debe retornar un arreglo  $R = [R[0], R[1], \dots, R[N-2]]$  de longitud  $N-1$ , un arreglo de padres correcto.

Para interactuar con la máquina, tu función puede llamar a las siguientes dos funciones:

La primera función es:

```
bool insert(int U, int X)
```

- $U$ : El índice de la hoja donde la bola debería ser insertada. Se debe satisfacer que  $0 \leq U < M$ .

- $X$ : El valor entero de la bola. Se debe satisfacer que  $0 \leq X \leq 1000$ .
- Esta función retorna `true` si la bola fue insertada satisfactoriamente, o `false` si la hoja  $U$  ya estaba ocupada.
- Esta función puede ser llamado a lo más 500 000 veces.

La segunda función es:

```
std::vector<int> collect()
```

- Almacena los valores de todas las bolas insertadas, vacía la máquina, y retorna los valores almacenados en el arreglo  $S$ .

Si en algún punto durante la ejecución de su programa el valor de  $C$  supera 1000, su solución recibirá el veredicto `Output isn't correct: Too many resources used`.

La estructura de la máquina es fija antes de que `find_structure` sea llamada. El calificador es **determinista** en el sentido de que, si se corre dos veces y ambas ejecuciones realizan la misma secuencia de operaciones, entonces los llamados a `collect` retornaran los mismos arreglos.

## Restricciones

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

## Subtareas

| Subtarea | Puntaje | Restricciones adicionales               |
|----------|---------|-----------------------------------------|
| 1        | 5       | La raíz contiene exactamente $M$ hijos. |
| 2        | 10      | $M \leq 3$                              |
| 3        | 25      | $N \leq 200, M \leq 45$                 |
| 4        | 60      | Sin restricciones adicionales.          |

En las subtareas 3 y 4, tu puntaje dependerá del valor de  $C$  como se describe a continuación:

### Subtarea 3 (25 points)

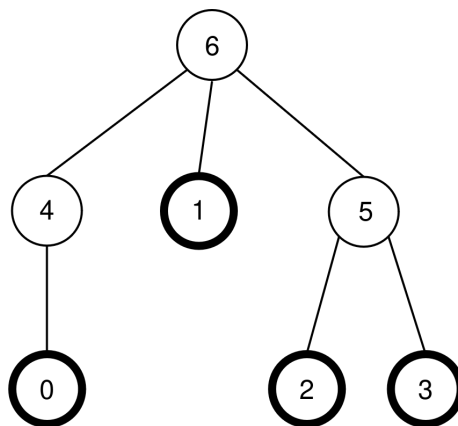
| Límites            | Puntaje |
|--------------------|---------|
| $1000 < C$         | 0       |
| $45 < C \leq 1000$ | 13      |
| $C \leq 45$        | 25      |

### Subtarea 4 (60 points)

| Límites             | Puntaje            |
|---------------------|--------------------|
| $1000 < C$          | 0                  |
| $200 < C \leq 1000$ | 7                  |
| $71 < C \leq 200$   | $47 - \frac{C}{5}$ |
| $44 < C \leq 71$    | $104 - C$          |
| $C \leq 44$         | 60                 |

## Ejemplo

Considere la misma máquina que se describió previamente en el enunciado del problema, de modo que  $N = 7$ ,  $M = 4$ , y  $P = [4, 6, 5, 5, 6, 6]$ . La máquina se muestra nuevamente en la siguiente figura:



El calificador hace el siguiente llamado:

```
find_structure(4)
```

La función puede hacer la siguiente secuencia de llamados:

1. **insert(0, 0)**: La hoja 0 está libre, por lo cual, una bola con valor 0 es colocada en la hoja 0. El nodo  $P[0] = 4$  está libre, así que la bola se mueve al nodo 4. El nodo  $P[4] = 6$  está libre, así que la bola se mueve al nodo 6. Como el nodo 6 es la raíz, el movimiento se detiene. El llamado retorna `true`.
2. **insert(3, 20)**: la hoja 3 está libre, por lo que se coloca una bola con valor 20 en la hoja 3. El nodo  $P[3] = 5$  está libre, por lo que la bola se mueve al nodo 5. Como  $P[5] = 6$  está ocupado, el movimiento se detiene. La llamada retorna `true`.
3. **insert(1, 10)**: la hoja 1 está libre, por lo que se coloca una bola con valor 10 en la hoja 1. Como  $P[1] = 6$  está ocupado, la bola permanece en el nodo 1. La llamada retorna `true`.
4. **insert(2, 30)**: la hoja 2 está libre, por lo que se coloca una bola con valor 30 en la hoja 2. Como  $P[2] = 5$  está ocupado, la bola permanece en el nodo 2. La llamada retorna `true`.
5. **insert(1, 25)**: la hoja 1 está ocupada, por lo que la bola no se coloca en la hoja 1. La llamada retorna `false`.
6. **insert(0, 20)**: la hoja 0 está libre, por lo que se coloca una bola con valor 20 en la hoja 0. El nodo  $P[0] = 4$  está libre, por lo que la bola se mueve al nodo 4. Como  $P[4] = 6$  está ocupado, el movimiento se detiene. La llamada retorna `true`.

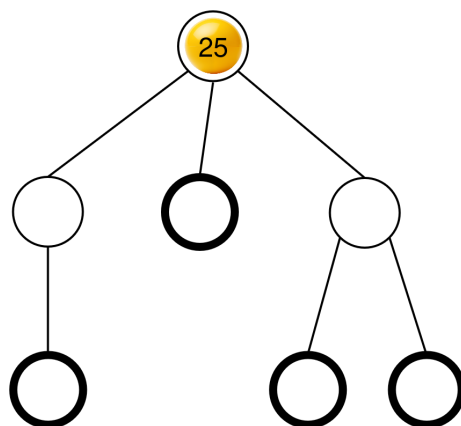
La configuración resultante de las bolas ya se mostraba en la descripción de la tarea.

A continuación, la función puede llamar a:

```
collect()
```

La ejecución de esta llamada ya se explicó en la descripción de la tarea, por lo que esta llamada retorna  $S = [0, 10, 20, 30, 20]$ . Después, la máquina vuelve a estar vacía.

A continuación, la función puede llamar a **insert(2, 25)**. La hoja 2 está libre, por lo que se coloca una bola con valor 25 en la hoja 0. Esta bola se mueve luego al nodo 5 y después al nodo 6, donde se detiene. La llamada retorna `true`. La configuración resultante de las bolas se muestra en la siguiente figura.



Finalmente, la función puede llamar a **collect()** que retorna  $S = [25]$  y vacía la máquina.

Hay dos matrices de padres correctas que `find_structure` puede devolver:  $R = P = [4, 6, 5, 5, 6, 6]$  (que corresponde al etiquetado  $L = [0, 1, 2, 3, 5, 4, 6]$ ). En este ejemplo,  $K = 2$  y  $B = 30$ , por lo que  $C = 32$ .

## Calificador de ejemplo

Formato de entrada:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato de salida:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aquí,  $Q$  es la longitud del arreglo  $R$  retornado por `find_structure`.